

Traplanner



Hệ thống lập kế hoạch lịch trình du lịch tự động bằng Multi-agent và Agentic AI

Người thực hiện Nguyễn Mạnh Hùng

Msv: B23DCCE040



Mục tiêu dự án



Traplanner ra đời để giải quyết bài toán trên thông qua:



Tự động hóa toàn phần

Tự động sinh lịch trình cá nhân hóa theo sở thích, ngân sách và thời gian.



Tối ưu hóa không gian

Tính toán lộ trình hợp lý nhất giữa các điểm đến, nhằm giảm thời gian tính toán cho người đi du lịch.



Giải quyết sự cố phát sinh

Cho phép chỉnh sửa lịch trình linh hoạt qua giao diện Chatbot thông minh.

.

Giới thiệu tổng quan về Traplanner



Hệ thống kết hợp sức mạnh của LLM và kiến trúc multiagent và agentic



Frontend

React + Vite, cung cấp giao diện Dashboard, Chatbot và Bản đồ tương tác trực quan (White-card aesthetic).



Backend

Python (Django), quản lý luồng xử lý Agent và API.



Database Architecture

Neo4j

Lưu trữ dữ liệu đồ thị (Graph) không gian và tuyến đường giao thông.

MongoDB

Lưu trữ dữ liệu cấu trúc (Document) về địa điểm, khách sạn, review, chi tiết lịch trình.

Tech Stack



AI & Logic

LangChain, LangGraph, LangSmith, LLM API (Gemini-flash-lite-2.5)

Database

MongoDB (Atlas), Neo4j (Docker), Redis (Cache/Session).

Backend

Python, Django Framework.

Frontend

React, Vite, Tailwind CSS / Vanilla CSS.

Kiến trúc Polyglot Synchronization



Tại sao lại dùng tới 2 cơ sở dữ liệu?



Vấn đề 1

MongoDB rất giỏi lấy thông tin chi tiết nhưng rất tệ trong việc tìm đường đi ngắn nhất.



Vấn đề 2

Việc call api google maps là tốt nhất nhưng chi phí đội lên rất cao và dễ dính limit, chỉ phù hợp khi tính tới yếu tố chi tiết như kẹt xe, ...



Giải pháp

Áp dụng Neo4j để xử lý quan hệ không gian, phù hợp với quy mô nhỏ.

Dữ liệu địa điểm được khởi tạo ở MongoDB. Một tiến trình Background (Asynchronous) sẽ đồng bộ tự động các tọa độ này sang Neo4j dưới dạng các Node và tạo Relationship (khoảng cách, thời gian di chuyển).

Quá trình tạo sinh lịch trình chuyến đi



Hệ thống chia nhỏ thành một Graph Pipeline với 6 Nodes

Extractor Trích xuất yêu cầu cốt lõi từ người dùng (Địa điểm, ngân sách, ngày đi).

Scheduler Sắp xếp các điểm đến thành một thời gian biểu chi tiết từng ngày/giờ.

Planner Lọc và chọn danh sách các điểm đến phù hợp (nơi lưu trú, hàng ăn, hoạt động tham quan/vui chơi)

Validation Kiểm tra ràng buộc logic (Giờ mở cửa, vượt ngân sách, khoảng cách phi lý) và sửa lỗi.

Mobility Tính toán lộ trình và phương tiện di chuyển

Generate Answer Tổng hợp toàn bộ dữ liệu lịch trình đã kiểm duyệt và chuyển thành văn bản phản hồi tự nhiên cho người dùng.

Extractor & Planner Nodes



Bước 1: Trích xuất yêu cầu và Lựa chọn điểm đến

Extractor Node

Chuyển đổi ngôn ngữ tự nhiên của người dùng (VD: "Tôi muốn đi Ninh Bình 2 ngày, ngân sách rẻ") thành dữ liệu JSON chuẩn (Địa điểm, số ngày, ngân sách, sở thích).

Planner Node

Đóng vai trò như "Người lên ý tưởng". Đầu tiên, nó truy vấn Neo4j để nội suy không gian (VD: Từ "Hà Nội" tìm ra các phường/xã trực thuộc). Sau đó, lấy danh sách này đi truy vấn MongoDB để bốc ra các Khách sạn, Quán ăn, và Chỗ vui chơi tại địa phương mà phù hợp nhất.

Mobility Node & Neo4j



Bước 2: Lập bản đồ và logic di chuyển

Ma trận di chuyển (Neo4j)

Cầm danh sách điểm đến lộn xộn từ Planner, Node này sẽ truy vấn Neo4j Graph để rút ra toàn bộ ma trận thời gian và khoảng cách (Từ A đi B mất bao lâu, B đi C mất bao lâu).

Thuật toán tối ưu (TSP)

Đóng vai trò như một "Tài xế bản địa". Nó tự động giải bài toán Người chào hàng (Traveling Salesperson Problem) để vẽ ra một đường đi mượt mà nhất, không bao giờ bị trùng lặp hay phải chạy xe lòng vòng quay ngược lại.

Dự toán chi phí

Ngoài việc tìm đường, Node này còn tính toán loại phương tiện phù hợp (VD: Xe máy, Taxi) và dự trù luôn tiền cước xe/xăng xe trước khi đẩy dữ liệu sang cho Scheduler.

Scheduler & Validation Nodes



Bước 3: Lên lịch trình và sửa lỗi

Scheduler Node

Dựa vào quỹ thời gian di chuyển do Mobility cung cấp, nó sẽ ráp các điểm đến thành một thời gian biểu chi tiết đến từng phút (VD: 08:00 - Tham quan Tràng An, 11:30 - Lên xe đi 15p, 11:45 - Ăn trưa). Quá trình này phải tuân thủ nghiêm ngặt ****Giờ mở/đóng cửa**** của từng địa điểm lấy từ MongoDB.

Validation Node

Điểm sáng của hệ thống Agentic là cơ chế tự sửa lỗi Node này đóng vai trò là chốt chặn cuối cùng kiểm tra toàn bộ logic: Lịch trình có bị lỗi ngân sách không? Có phi lý kiểu "ăn trưa lúc 4h chiều" hay "dịch chuyển 50km trong 5 phút" không? Nếu có lỗi, nó sẽ tự động bỏ bản nháp và ép các node trước đó phải sửa lại cho đến khi hoàn hảo mới thôi.

Generate Answer & UI



Bước 4: Tổng hợp dữ liệu và tương tác người dùng

Generate Answer Node

Nhận dữ liệu JSON lịch trình đã qua kiểm duyệt từ Validation Node, sau đó sử dụng LLM để biên dịch thành văn bản câu trả lời (định dạng Markdown) gửi về Frontend.

Giao diện Trip Day Detail

Hiển thị chi tiết lịch trình theo từng ngày. Frontend dùng `locationId` để khớp với MongoDB, từ đó lấy chính xác hình ảnh (`img\_url`), giá cả và mô tả để hiển thị lên UI.

Tính năng cập nhật cục bộ

Khi người dùng sử dụng tính năng chat để "Đổi quán ăn khác", Chatbot chỉ gọi API để thay thế đúng quán ăn đó trong mảng dữ liệu, giữ nguyên các địa điểm và giờ giấc còn lại để tiết kiệm tài nguyên.

Đánh giá và tối ưu hiệu năng



Bằng việc sử dụng Langsmith làm phép đo:

Theo dõi luồng chạy (Tracing)

Tích hợp nền tảng LangSmith để ghi log trực tiếp toàn bộ quá trình thực thi của LangGraph (Đo đạc Input/Output, lượng Token tiêu thụ và thời gian phản hồi của từng Agent).

Phân tích độ trễ (Latency Bottleneck)

Nhờ LangSmith, em đã phát hiện ra điểm yếu của hiệu năng nằm ở việc lạm dụng LLM cho các tác vụ đơn giản (Ví dụ: Gọi LLM để tính toán khoảng cách hoặc cộng trừ ngân sách).

Giải pháp tối ưu

Chuyển các khâu kiểm tra logic cứng (giờ mở cửa, chi phí) sang mã code Python truyền thống. Phương pháp này có nhược điểm là sinh ra lịch trình không giống nhịp sinh hoạt tự nhiên của con người nên cần cải thiện trong tương lai

Kết luận



- Dự án Traplanner đã chứng minh tính khả thi của việc kết hợp Multi-Agent System và đa cơ sở dữ liệu trong bài toán thực tế.
- Thành công nâng cấp LLM từ một mô hình "hỏi-đáp" thụ động sang một hệ thống mang tính chất Agentic tự chủ: Có khả năng tự phân chia công việc cho các node, tự truy xuất dữ liệu, suy luận logic và đặc biệt là năng lực tự động kiểm duyệt, sửa lỗi
- Việc chia nhỏ thành nhiều agent khiến hệ thống phải gọi LLM API nhiều lần, dẫn đến độ trễ cao và tốn kém chi phí vận hành. Ngoài ra, nguồn dữ liệu hiện tại đang đóng gói tại số ít địa điểm nên chưa thể bao quát các tình huống du lịch thời gian thực.



THANKS FOR LISTENING !!!